

PSTU National Hackathon 2026

Final Round Problem Statement - Recovered from Public Sources

Important: I could not find the original final-round email/PDF publicly indexed. This document reconstructs the challenge from the official PSTU event page, a public finalist GitHub repository containing an approved *problem.md*, and participant write-ups. Core requirements are strongly supported; items marked "participant-described" are reported by a runner-up team but are not present in the normalized GitHub problem file.

1. Final Round Context

Event	PSTU IT Carnival 2026 - National Hackathon
Final date	29 August 2026
Venue	TSC Conference Hall, Patuakhali Science and Technology University
Problem delivery	Final problem statement emailed to selected teams at 8:00 AM
Build window	9:00 AM to 3:00 PM (6 hours)
Evaluation	Mid evaluation around 11:00 AM; final demonstration/jury evaluation 3:00 PM to 5:00 PM
Format	On-site industry-level software engineering challenge

2. Recovered Final Challenge: Money Movement Application

Objective: Build a user-facing, closed-ecosystem application for simulated BDT money movement. Registered users should be able to hold a demo balance, send money to another valid user, request money, view pending requests addressed to them, and fulfill those requests while keeping balances and request states persistent and correct.

The system is a hackathon MVP using fake/simulated money only. Real banks, cards, payment gateways, deposits, withdrawals, settlement, KYC integrations, and real-money custody are outside the required core MVP.

Core functional requirements

- **Account creation:** allow users to register/create an application account.
- **Demo starting balance:** a public finalist implementation normalized the requirement as BDT 100,000 simulated balance for each newly created user.
- **Balance view:** users can view the current persisted simulated balance.
- **Send money:** a user can send a valid amount to another valid application user.
- **Request money:** a user can create a money request directed to another valid user; creating the request must not move money.
- **Pending requests:** the designated payer can see valid pending requests addressed to them.
- **Fulfill request:** fulfilling a pending request moves the corresponding amount from payer to requester and completes the request.
- **Persistence:** account state, balances, requests, and completed state must remain correct after refresh/read-back.

Correctness and backend expectations

- The backend must be authoritative for money movement and request-state changes.

- Reject invalid users, invalid amounts, insufficient funds, and invalid request states.
- Money movement must be atomic: debit and credit both happen, or neither happens.
- Duplicate submissions/retries must not move the same money twice.
- Concurrent sends or fulfillments must not cause double-spending or negative accepted balances.
- A pending request does not change balances; a completed request must not be fulfilled again.
- Rejected operations must not partially modify balances or request state.
- Total simulated value should remain conserved after account seeding, except for the initial demo-balance grant on account creation.

3. Participant-Described Extended Scope

A 1st runner-up team publicly described the final challenge as an enterprise-grade digital money-movement platform for a high-concurrency fintech scenario. Their write-up explicitly mentions the following challenge scope. These details are useful, but they are **participant-described** rather than independently confirmed from the normalized *problem.md*.

- Peer-to-peer transfers
- Smart due-date requests
- Multi-party bill splitting
- ~~High-concurrency correctness~~ under intense hackathon time constraints

Practical interpretation: If you are preparing to reproduce the final question for practice, implement the confirmed core first (account, balance, send, request, fulfill, persistence, atomicity/concurrency) and then add due-date requests and bill splitting as extended features.

4. Golden Path / Demo Scenario

1. Alice registers and receives simulated BDT 100,000.
2. Bob registers and receives simulated BDT 100,000.
3. Alice sends BDT 2,500 to Bob.
4. The transfer succeeds exactly once; resulting balances are correct.
5. Bob requests BDT 1,200 from Alice; the request becomes pending and balances do not change.
6. Alice sees the pending request and fulfills it.
7. BDT 1,200 moves exactly once from Alice to Bob and the request becomes completed.
8. A refresh/read-back shows the same persisted balances and request state.
9. At least one important invalid, duplicate, retry, or concurrent operation is rejected/handled without corrupting state.

5. Suggested Acceptance Checklist

- [] User registration works.
- [] Starting demo balance is visible.
- [] Valid P2P transfer works and persists.
- [] Insufficient-funds transfer fails without partial changes.
- [] Money request creation works without moving funds.
- [] Only the designated payer can fulfill a valid pending request.
- [] Fulfillment moves money once and marks the request completed.
- [] Duplicate/retry attempts do not duplicate financial effects.
- [] Concurrent operations do not double-spend balances.
- [] Refresh/read-back preserves correct state.
- [] Optional extension: due-date requests.
- [] Optional/participant-described extension: multi-party bill splitting.

6. Public Sources Used

Official PSTU IT Carnival 2026 Hackathon page

Confirms final date, on-site format, 8:00 AM problem email, 6-hour build window, and final evaluation schedule.

<https://pstu-it-carnival.vercel.app/events/hackathon>

ZenoxXYZ/PSTU-HACKATHON-2026 - problem.md

Strongest public source for the normalized final Money Movement Application requirements, core workflows, correctness rules, persistence, and golden path.

<https://github.com/ZenoxXYZ/PSTU-HACKATHON-2026/blob/main/problem.md>

ZenoxXYZ/PSTU-HACKATHON-2026 - README

Confirms the project was built for PSTU National Hackathon 2026 and implements the same money-movement scope.

<https://github.com/ZenoxXYZ/PSTU-HACKATHON-2026>

ahsandevhub/bangla-pay

Independent PSTU National Hackathon 2026 solution implementing a simulated mobile financial service with fake BDT wallets, transfers, payment requests, receipts, and correctness-focused architecture.

<https://github.com/ahsandevhub/bangla-pay>

Farhan Zaman / Mushfiqur Rahman participant write-up (LinkedIn, indexed publicly)

1st runner-up write-up describes the challenge as peer-to-peer transfers, smart due-date requests, and multi-party bill splitting in a high-concurrency fintech ecosystem.

<https://bd.linkedin.com/in/mushfique693>

Abil Waquer Zihan participant write-up (LinkedIn, indexed publicly)

Describes the PSTU Hackathon 2026 Money Movement Application and links the public GitHub repository.

https://www.linkedin.com/posts/abil-waquer-zihan-941798220_github-zenoxxyzpstu-hackathon-2026-activity-7500939990927839232-HohF

Compilation note

Compiled on 28 September 2026 from public web and GitHub sources. This is a reconstruction for study/reference, not an organizer-issued original PDF. If the original emailed question PDF becomes public, it should take precedence over this reconstruction.